I want to build an AI agent today (full course):

> 📎 来源: https://x.com/hooeem/article/2037250422403113188

No-one has made a full course so that anyone (yes, you) can create an AI agent from scratch.

If you wanted to, you could read this article and create an agent that is useful for you to utilise today, because creating an agent for agents sake means nothing, it needs to be for a reason.

So what did I do?

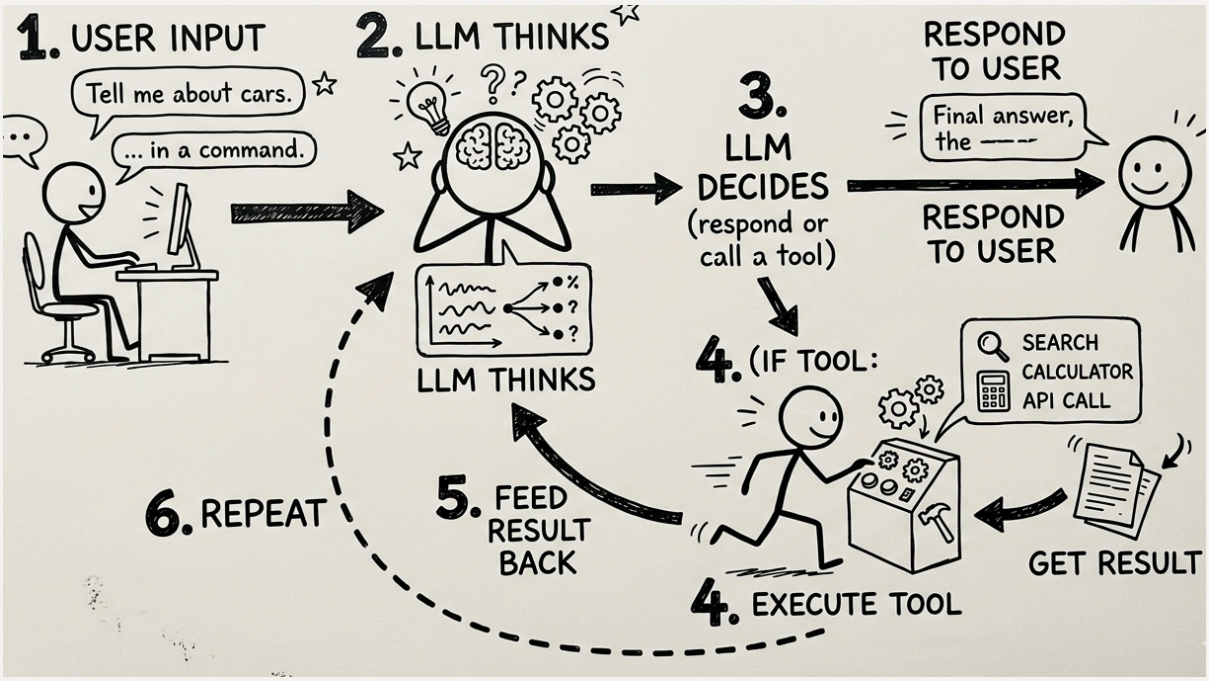
I took resources from Anthropic, OpenAI, and other experts on the internet who have given bits of information that is useful here and there, I took them all, put it together with my mate Claude, and created a full course for the layman (me) to understand so that we (me and you) can create an agent today.

This is a long article, at the end of it, you will be able to build your first agent, just so to help you navigate this article the text that is **CAPITALISED AND BOLD** are the subheadings, there's 8 in total, each one will have an image so you can get to each part you want to:

1. How agents work
2. Five workflows
3. Building your agent
4. Utilising tools
5. Giving your agent memory
6. Making your agent work
7. Multiple agents
8. Wrapping it all up

Okay, let's get straight into it here...

1: HOW AGENTS WORK



It's important to know this stuff, if you don't then you'll have no idea why you'll need one or not... so...

This is the core loop shared by all agents:

User input → LLM thinks → LLM decides (respond or call a tool) → if tool: execute it, feed result back → repeat

The LLM is the **"brain"** that reasons. **Tools** are the "hands" that perform actions (calculator, web search, file I/O). **Memory** is the "notepad" that records what has happened so far. Whether you use LangGraph, CrewAI, Anthropic's SDK or OpenAI's Agents SDK, the frameworks wrap this loop with abstractions but do not change its essence.

Augmented LLMs

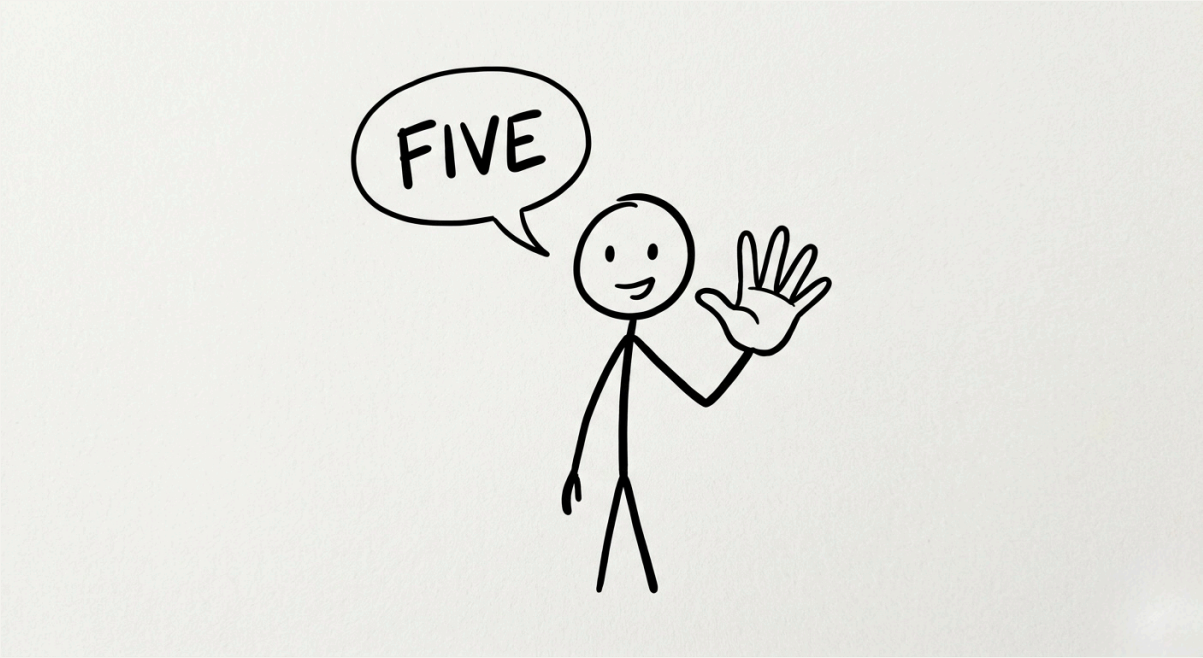
A plain LLM accepts text and emits text. An **augmented LLM** adds three capabilities:

- **Tools:** functions the model can call (calculators, databases, APIs, file operations, etc.). Anthropic and OpenAI expose tools via JSON schemas; Anthropic passes an input_schema while OpenAI wraps functions in a function object with parameters
- **Retrieval:** ability to pull relevant information from external sources (search engines, documents, vector databases).
- **Memory:** ability to retain information across interactions via a message history or other persistent storage.

Workflows vs. true agents

The distinction between **workflows** and **agents** matters when choosing an approach. Workflows are deterministic; your code controls execution and the same input always produces the same path. They are ideal for well-defined tasks with fixed steps and are cheaper (fewer LLM calls). Agents are dynamic; the LLM decides the next step and may call tools repeatedly. They are best for open-ended tasks but cost more. The process for you finding if you need to create an agent or not should start by using a simple workflow and then seeing whether or not you'll graduate that to become an autonomous agent.

2: THE FIVE CORE WORKFLOW PATTERNS



Because believe it or not, most problems can actually be solved without needing full autonomy. These five patterns, documented by Anthropic and widely adopted, cover common cases. Each pattern relies on an augmented LLM.

> ***Pattern 1: Prompt chaining***

What it is: Break a task into sequential steps. Each LLM call processes the output of the previous one. Add programmatic "gates" between steps to verify quality.

When to use it: Tasks that decompose cleanly into fixed subtasks. You trade speed for accuracy by making each LLM call simpler.

Example use cases: Generate marketing copy then translate it. Write an outline, verify it covers key topics, then write the full document.

> *Pattern 2: Routing*

What it is: Classify incoming input, then route it to a specialised handler. Each handler gets its own optimised prompt.

When to use it: Different categories of input need fundamentally different treatment. Customer service triage is the classic example.

> *Pattern 3: Parallelisation*

What it is: Run multiple LLM calls simultaneously. **Sectioning** splits a task into independent subtasks processed in parallel. Voting runs the same task multiple times and aggregates results for higher confidence.

When to use it: When subtasks are independent (sectioning) or when you need consensus on a critical decision (voting).

> *Pattern 4: Orchestrator-workers*

What it is: A central LLM (the orchestrator) dynamically breaks down a task and delegates subtasks to worker LLMs. Unlike parallelisation, the subtasks are not predefined, the orchestrator decides them at runtime.

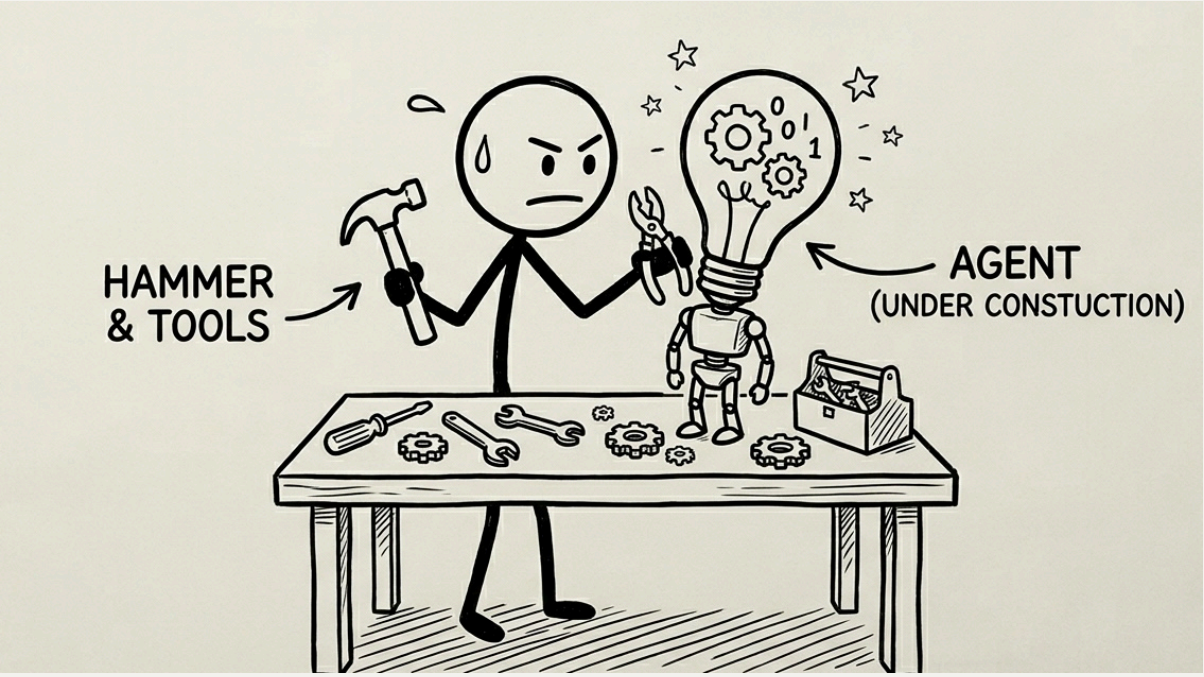
When to use it: Complex tasks where you cannot predict the structure in advance. Code generation across multiple files, research tasks, and report writing.

> *Pattern 5: Evaluator-optimiser*

What it is: One LLM generates output, another evaluates it and provides feedback. If evaluation fails, the feedback loops back. This repeats until quality criteria are met.

When to use it: When clear evaluation criteria exist and iterative refinement adds measurable value. Translation, code generation, and writing tasks.

3: BUILDING YOUR AGENT



This is the part of the article you came for... let's dive in:

So how do you turn "I want an agent to do XYZ" into something real?

The easiest way to think about it is this:

1. **Write down the job**
2. **Decide what tools it needs**
3. **Tell the model how to behave**
4. **Test it on 5 real examples**
5. **Only add more complexity if it fails**

You do **not** need to master five frameworks to build your first agent. For me and you the best starting point is:

- **Anthropic** if you want an agent that works like a capable operator with tools, files, shell commands, web actions, and strong coding workflows
- **OpenAI** if you want a clean developer SDK with hosted tools, handoffs, guardrails, and a simple path to production

This guide focuses mainly on those two.

The simplest mental model

When building an agent, answer these four questions first:

1. What is the outcome? What should the agent actually produce?

Examples:

- “Research a topic and write a summary”
- “Read my notes and turn them into flashcards”
- “Look at support requests and route them correctly”
- “Compare products and give me the best option”
- “Review my content and rewrite it in my voice”

2. What information does it need? Does it need web search, files, a database, a spreadsheet, a CRM, or just the user’s message?

3. What actions should it be allowed to take? Can it only answer? Can it search? Can it edit files? Can it send emails? Can it write code? Can it call your own functions?

4. What rules must it follow? Tone, format, constraints, safety rules, what to do when uncertain, and what “good” looks like.

If you can answer those four questions clearly, you can usually build the first version of your agent in a day.

> *Quick hack we'll dive into shortly, you can take your idea, give it to your LLM, ask it to think deeply, let it answer all the above questions for you.*

How to use AI itself to design the agent before you build it

A very practical move is to use Claude or ChatGPT **before coding** to help you define the agent.

Paste something like this:

```
I want to build an AI agent.

My goal:
[describe what you want it to do]

The user will ask things like:
[add 5 realistic examples]

The agent should have access to:
[web search / files / calculator / custom API / nothing else]
```

```
It must always:
[list non-negotiable rules]

It must never:
[list boundaries]

Please turn this into:
1. A clear agent spec
2. A system prompt
3. A tool list
4. A first version roadmap
5. 10 test cases
```

That one prompt can help a beginner turn a vague idea into a buildable plan.

A beginner-friendly formula for agent design

Use this structure every time:

Agent = Role + Goal + Tools + Rules + Output format

Example:

- **Role:** Research assistant for crypto projects
- **Goal:** Find accurate information and summarise it clearly
- **Tools:** Web search, file search, calculator
- **Rules:** Cite sources, do not guess, flag uncertainty
- **Output format:** Summary, risks, opportunities, final verdict

That is the foundation of most useful agents.

Start with one of these five beginner agent types:

If you are new, do not start by building a multi-agent swarm. Start with one of these:

1. Research agent

Use when you want the agent to gather information and summarise it.

Examples:

- “Research the best rehab exercises for ankle sprain”
- “Find the latest updates on a crypto protocol”
- “Compare three laptops”

Needs:

- Web search
- File search if you want it to use your own documents
- Clear output format

2. Content agent

Use when you want the agent to write, rewrite, summarise, or transform content.

Examples:

- “Turn my notes into a newsletter”
- “Rewrite this in my brand voice”
- “Summarise this meeting transcript”

Needs:

- Usually just a strong system prompt
- Optional file access
- Examples of your preferred style

3. Workflow agent

Use when you want the agent to follow a repeatable business process.

Examples:

- “Classify support tickets”
- “Route leads to the right category”
- “Check form submissions and create a response draft”

Needs:

- Clear categories
- Rules

- Sometimes custom tools or API calls

4. Personal knowledge agent

Use when you want the agent to answer questions using your documents.

Examples:

- “Answer using my PDFs only”
- “Search my notes and explain this topic”
- “Find all references to this client”

Needs:

- File search or RAG
- Clear instruction to stay grounded in provided material

5. Operator agent

Use when you want the agent to take actions in an environment.

Examples:

- “Read these files and edit them”
- “Search the web, gather findings, and save a report”
- “Run shell commands and help me debug code”

Needs:

- Tools
- Permissions
- Strong safety boundaries

Anthropic: the easiest way to think about building your first agent

Anthropic’s agent tooling is especially helpful when you want the model to **use tools and operate in an environment**. Claude Code launched in February 2025, and the Claude Code SDK was later renamed the Claude Agent SDK in September 2025. The current GitHub release listed in March 2026 is v0.1.50.

When Anthropic is a good choice

Choose Anthropic first if you want an agent that should:

- read, write, and edit files
- use shell commands
- search the web
- use MCP tools
- work well for coding and technical tasks
- feel like a capable assistant operating step by step

What you are really doing with Anthropic

At a beginner level, you are doing three things:

1. Giving Claude a **job**
2. Giving Claude **tools**
3. Letting Claude loop until the task is done

That is all.

Beginner example: a research-and-summary agent

Let's say you want:

> *"An agent that researches a topic and writes me a clean report."*

Your build plan would be:

- **Role:** Senior research assistant
- **Goal:** Find accurate information and summarise it clearly
- **Tools:** Web search, maybe file access
- **Rules:** Cite sources, say when uncertain, keep it concise
- **Output:** Bullet summary + key risks + conclusion

That becomes your system prompt:

```
SYSTEM_PROMPT = '''
You are a careful research assistant.

Your job is to help the user research topics accurately.
Use tools when needed.
Do not guess.
If information is uncertain or incomplete, say so clearly.
Always produce:
1. Summary
2. Key findings
3. Risks or uncertainty
4. Final conclusion
'''
```

Now the user can ask:

- “Research the latest AI agent SDKs”
- “Compare Anthropic and OpenAI for building a beginner agent”
- “Find three strong sources and summarise them”

That is already a real agent.

Beginner example: a file-based writing agent

Maybe you want:

> *“Read my notes and rewrite them into a clean article in my voice.”*

Then your design becomes:

- **Role:** Writing assistant
- **Goal:** Turn rough notes into polished writing
- **Tools:** File read, maybe file write
- **Rules:** Preserve meaning, improve clarity, match tone
- **Output:** Final article + optional title ideas

That is much easier to build than a vague “content agent”.

What you should ask AI before building the Anthropic agent:

Use your LLM to help you define the build:

```
Help me design an Anthropic agent.

My goal is:
[goal]

I want the agent to be able to:
[list actions]

I want the agent to use these tools:
[list tools]

I want the final output to look like:
[format]

Please create:
1. A strong system prompt
2. A minimal tool list
3. A first version Python example
4. 10 test prompts
5. Suggestions to improve reliability
```

That prompt will usually get you 80% of the way there.

OpenAI: the easiest way to think about building your first agent

OpenAI launched its Agents SDK on 11 March 2025 alongside the Responses API and built-in tools for web search, file search, and computer use. The Python package openai-agents was at version 0.13.1 in March 2026.

When OpenAI is a good choice

Choose OpenAI first if you want:

- a very clean agent API
- easy custom function tools
- built-in hosted tools
- handoffs between specialist agents
- guardrails and tracing
- a smooth path from prototype to production

What you are really doing with OpenAI

At a beginner level, the build is:

1. Create an **Agent**
2. Give it **instructions**
3. Add **tools** if needed
4. Run it with a real user request

That is it.

Beginner example: a support triage agent

Suppose your goal is:

> *"Read incoming support requests and decide whether they are billing, technical, or sales."*

That becomes:

- **Role:** Support triage assistant
- **Goal:** Categorise requests correctly
- **Tools:** None, maybe later a CRM tool
- **Rules:** Choose one category only, explain briefly
- **Output:** Category + reason

This would look like this:

```
from agents import Agent, Runner

agent = Agent(
    name="Support Triage Agent",
    instructions=\"\"\"
You classify customer requests.
Choose exactly one category:
- billing
- technical
- sales

Reply with:
1. Category
```

```
2. One sentence explaining why
\"\"\",
)

result = Runner.run_sync(agent, "I was charged twice for my
subscription this month.")
print(result.final_output)
```

That is already a useful agent.

Beginner example: adding a custom tool

Now suppose you want:

> *"Calculate values for the user when needed."*

```
from agents import Agent, Runner, function_tool

@function_tool
def calculate(expression: str) -> str:
    import math
    allowed = {k: v for k, v in math.__dict__.items() if not
k.startswith("__")}
    return str(eval(expression, {"__builtins__": {}}, allowed))

agent = Agent(
    name="Math Helper",
    instructions="Help the user solve maths problems. Use the
calculator tool when needed.",
    tools=[calculate],
)

result = Runner.run_sync(agent, "What is compound growth on 10000
at 5 percent for 8 years?")
print(result.final_output)
```

Now the agent is not just chatting. It is taking actions through a tool.

Beginner example: using hosted tools

The OpenAI Agents SDK also supports hosted tools like web search, file search, and code interpreter through helper functions in the SDK docs. A beginner can think of these as "prebuilt capabilities" you attach to the agent instead of writing everything from scratch.

That means you can build agents like:

- "Research this topic from the web and summarise it"
- "Search my files and answer from them"
- "Run code to analyse this data"

What you should ask your LLM before building the OpenAI agent:

```
Help me design an OpenAI agent.

My goal:
[goal]

The tasks I want it to handle:
[list tasks]

The tools I think it needs:
[list tools]

The output should look like:
[format]

Please give me:
1. A clear agent instruction block
2. The simplest first version
3. A version with tools if needed
4. 10 test prompts
5. Common failure modes and how to fix them
```

How to customise your agent so it actually does what you want

This is where beginners usually go wrong. They build a generic assistant instead of a specific agent.

Use this checklist.

1. Make the job narrow

Bad:

- “Help with business stuff”

Good:

- “Summarise sales calls into action points”
- “Categorise leads into hot, warm, cold”
- “Research crypto projects and output risks, catalysts, and verdict”

2. Define the output format

Bad:

- “Give me an answer”

Good:

- “Return: Summary, evidence, risks, next steps”
- “Return JSON with category, confidence, explanation”
- “Return a bullet list under 5 headings”

3. Give examples

If you want tone, structure, or classification quality, examples help a lot.

Tell the model:

- “Here are 3 examples of good outputs”
- “Here are 5 examples of how to classify requests”
- “Write in this exact style”

4. Add tools only when needed

Do not add web search if the task is just rewriting notes. Do not add file access if the answer should come from the prompt alone. Every extra tool adds complexity.

5. Test with real prompts, not ideal ones

Use messy prompts like a real user would type.

Instead of testing only:

- “Please classify this technical issue”

Also test:

- “my account is broken and i keep getting charged what do i do”

That is where you learn what your agent actually does.

Here's your build path:

Step 1: Write one sentence describing the agent Example: “I want an agent that turns my rough notes into a clean weekly newsletter.”

Step 2: Ask Claude or ChatGPT to turn that into:

- an agent spec
- a system prompt
- a tool list
- 10 test prompts

Step 3: Build the smallest working version No multi-agent setup. No complex memory. No RAG unless needed.

Step 4: Test it on 10 real examples

Step 5: Improve one thing at a time

- prompt
- output structure
- examples
- tools
- memory
- retrieval

That order matters. Don't get bogged down by it all.

Avoid this mistake:

The biggest mistake is trying to build an “all-purpose super agent”.

Do not start with:

- web search
- file search
- database access
- memory
- multi-agent handoffs
- complex guardrails
- custom dashboards
- 20 tools

Start with:

- one job
- one agent
- one clear prompt
- one or two tools maximum
- five to ten real test cases

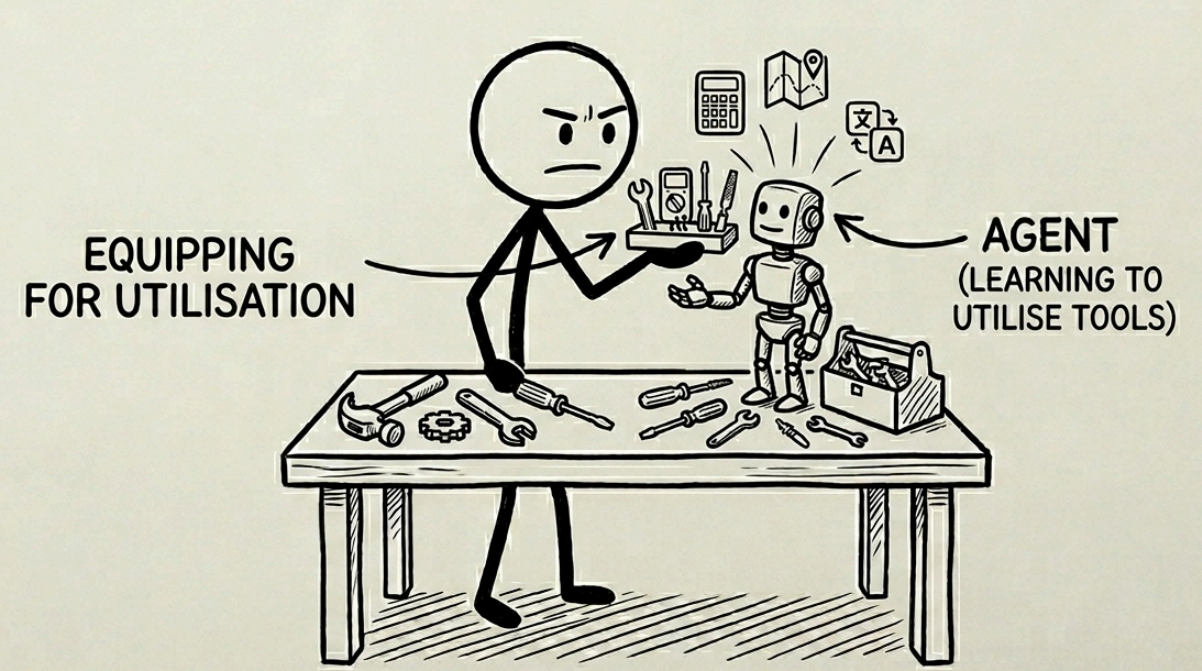
This is how you will succeed, by not overcomplicating it for yourself.

Practical takeaway:

You're at the end of part 3 now, this was the section that is teaching you how to build your first agent, at the end of this section you should be able to say:

- I know what my agent is for
- I know what tools it needs
- I know what rules it should follow
- I know how the output should look
- I know whether to start with Anthropic or OpenAI
- I know how to use AI itself to help me design the first version

4: UTILISING TOOLS



Most people get this wrong.

They think:

> *“More tools = smarter agent”*

Wrong.

Better tools = smarter agent. Fewer tools = more reliable agent.

The simplest way to think about tools

A tool is just:

> *“Something the AI can’t do on its own”*

Examples:

- calculate numbers
- search the web
- read your files
- send an email
- query a database

Step 1: Ask yourself: "Does this need a tool?"

Before adding anything, ask:

- Can the model answer this using just reasoning?
- Or does it need real-world data or actions?

Example:

No tool needed:

- “Rewrite this email”
- “Summarise this text”
- “Explain this concept”

Tool needed:

- “What’s the weather right now?”
- “Search the latest news”
- “Calculate compound interest”
- “Pull data from my spreadsheet”

👉 Rule:

> *If it requires **external data or action** → **use a tool** If not → don’t add one*

Step 2: Use AI to help you with your tools:

```
I am building an AI agent.

My goal:
[describe goal]

Here is what I think the agent needs to do:
[list actions]

Which of these require tools?
What tools should I create?
Keep them simple and minimal.

Return:
1. Tool list
2. Tool descriptions
```

```
3. Inputs required for each tool
```

This will save you a lot of time.

Step 3: Keep it simple stupid

Bad tool:

```
manage_files(action, file, destination, overwrite, format,
permissions)
```

Good tools:

```
read_file(path)
write_file(path, content)
delete_file(path)
```

👉 Rule:

> *One tool = one clear job*

Step 4: Tell the agent WHEN to use the tool

This is where most people fail.

Bad:

> *“Calculator tool”*

Good:

> *“Use this tool whenever maths is required. Never guess calculations.”*

Step 5: Let the agent fail and fix it

Run real tests like:

- “what’s 2^16”
- “calculate 7% growth over 10 years”

If it:

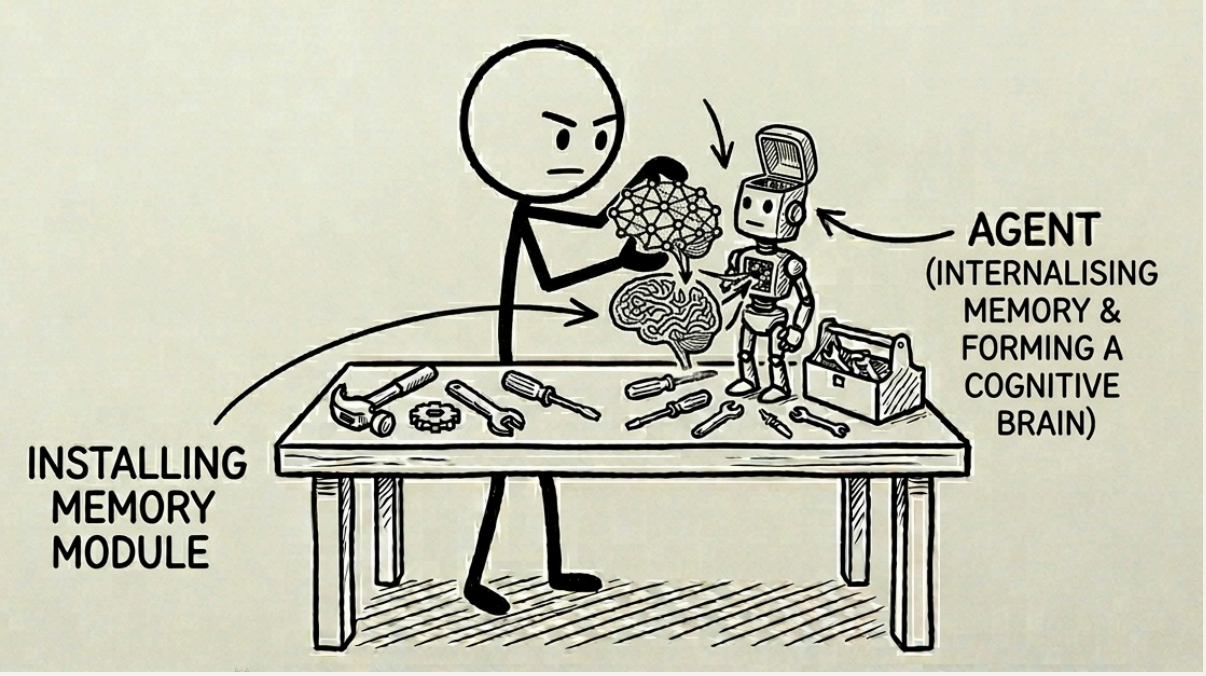
- doesn’t use the tool → fix description
- uses it incorrectly → fix inputs
- hallucinates → make rules stricter

You're at the end of part 4 now, you should know:

- You don’t need many tools
- You can use AI to design them
- Simpler tools = better agents
- Tool instructions matter more than the tool itself

Okay, moving on…

5: GIVE YOUR AGENT MEMORY



People massively overcomplicate this.

You only need to understand this:

> *There are TWO types of memory*

1. Short-term memory (conversation)

This is just:

> *“What has been said so far”*

You already get this by default.

2. Long-term memory (external knowledge)

This is:

> *“Stuff the agent can look up later”*

Examples:

- your notes
- PDFs
- documents
- databases

When do you ACTUALLY need memory?

Ask:

- Does the agent need to remember things across messages? → yes → short-term
- Does it need to use external documents? → yes → long-term
- Otherwise → you probably don’t need it

Step 1: Let AI help you decide if you need it

```
I am building an AI agent.

My goal:
[goal]

Does this agent need:
1. Conversation memory?
2. External knowledge (RAG)?

If yes, explain why.
If no, explain why not.

Keep it simple.
```

Step 2: You have three options...

Option A: No memory (start here)

- Best for most beginners
- Works for 70% of use cases

Option B: Conversation memory

- Already handled in most SDKs
- Just don’t reset messages

Option C: File-based memory (easy RAG)

- Upload documents
- Use file search tool

Step 3: Don't go full retard (overdo it)

Big mistake:

- adding vector DB
- embeddings
- complex pipelines

before you even know if you need them

👉 Rule:

> *If your agent works without memory → don’t add it*

Okay, you're at the end of part 5, now you should know:

- Most agents don’t need complex memory
- Start simple
- Add memory only when something breaks

6: MAKING YOUR AGENT WORK IRL

This is where agents end up either being shit, or goatee, and a lot of them are shit because of:

- bad prompts
- no testing
- unrealistic expectations

so...

Step 1: Use AI to create test cases

```
I built an AI agent with this goal:
[goal]

Create 15 realistic user inputs:
- messy
- vague
- real-world style

Also include:
- edge cases
- confusing inputs
- bad inputs
```

Step 2: Test like a real user

Don’t test:

> *“Please classify this billing request”*

Test:

> *“why tf did i get charged again”*

Step 3: Fix one thing at a time

When it fails, ask:

- Is the prompt unclear?
- Is the output format vague?
- Is a tool missing?
- Is a rule missing?

Step 4: Use AI to debug your agent

```
Here is my agent:

Here is what I asked:
[input]

Here is the output:
[output]

What went wrong?
How do I fix it?
Be specific.
```

Step 5: Don’t go crazy too early

Do NOT add:

- multiple agents
- complex workflows
- automation pipelines

until:

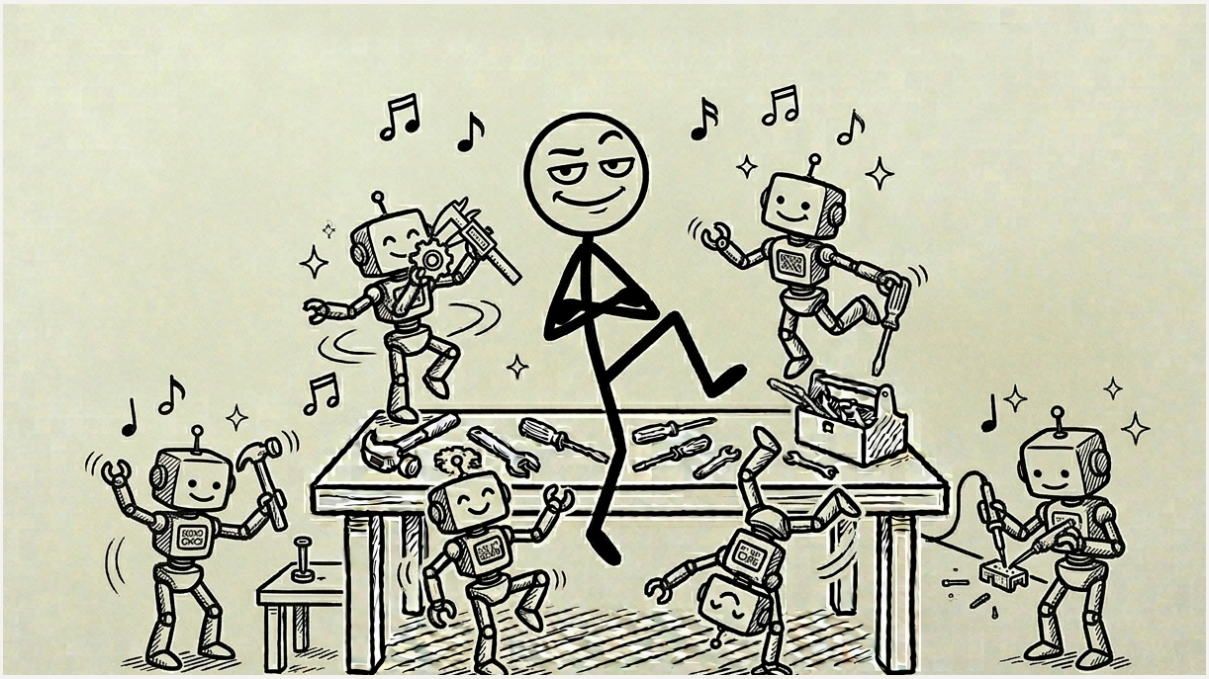
- your simple version works consistently

You're at the end of part 6, you should now know:

- Testing is everything
- AI can help you debug itself
- Fix clarity before adding complexity

NEXT...

7: MULTIPLE AGENTS



You can go completely off track here easily.

People think:

> *"More agents = more powerful"*

Wrong.

Start with ONE agent

Always.

Only add more when:

- the task is clearly split
- one agent is struggling
- roles are very different

The only 3 times you need multiple agents

1. Different skills

Example:

- Research agent
- Writing agent

2. Clear pipeline

Example:

- Input → Analyse → Write → Output

3. Different permissions

Example:

- One agent can read data
- One agent can execute actions

Step 1: Use AI to decide if you need multiple agents

```
I built an AI agent.

Here is its job:
[describe]

Should this be:
1. A single agent
2. Multiple agents

If multiple:
- what roles?
- why?

Keep it simple.
```

The safest pattern to use:

Supervisor model:

User → Main agent → (calls others if needed)

Do NOT start with:

- swarm
- fully autonomous multi-agent systems

They break easily.

Step 2: Keep roles simple stupid

Bad:

- “AI strategist agent with dynamic cognitive layering”

Good:

- “Research agent”
- “Writer agent”

Step 3: Add agents slowly

Start:

- 1 agent

Then:

- 2 agents max

Only expand if:

- you see real benefit

The takeaway for part 7?

- Most people do NOT need multiple agents
- Single agent + good tools = enough

- Add complexity only when forced

8: WRAPPING THIS ARTICLE UP!

The most important insight from this guide is that **agents are conceptually simple but operationally demanding**. The core loop, LLM thinks, calls tools, repeats, fits in 50 lines of Python. The real work is in tool design, error handling, evaluation, and knowing when simpler patterns (prompt chaining, routing) will outperform autonomous agents.

Three actionable takeaways for getting started:

1. **Build the from-scratch agent first**. Understanding the raw loop makes every framework transparent rather than magical. You will debug issues faster and choose tools more wisely.
2. **Start with the simplest pattern that works.** A prompt chain handles most multi-step tasks. A routing pattern handles most classification-then-action workflows. Graduate to autonomous agents only when you need the LLM to decide the execution path dynamically.
3. **Invest in tool design and evaluation early.** Well-designed tools with clear names, precise descriptions, and structured error messages will improve agent performance more than switching models or frameworks. And 20 good test cases will catch more bugs than any amount of manual testing.

The field is moving fast, MCP became a universal standard in under a year, both major providers shipped Agent SDKs, and new frameworks appear monthly. But the fundamentals in this guide are stable: the agentic loop, the five workflow patterns, the principles of good tool design, and the discipline of starting simple. Master these, and you can adapt to whatever comes next.

YOU CAN NOW BUILD AN AGENT.

and finally...

A LITTLE PLUG TO MY NEWSLETTER:

> *Mar 26, 2023*
>
> *http://sevenc.substack.com sign up for free*

let's cook...